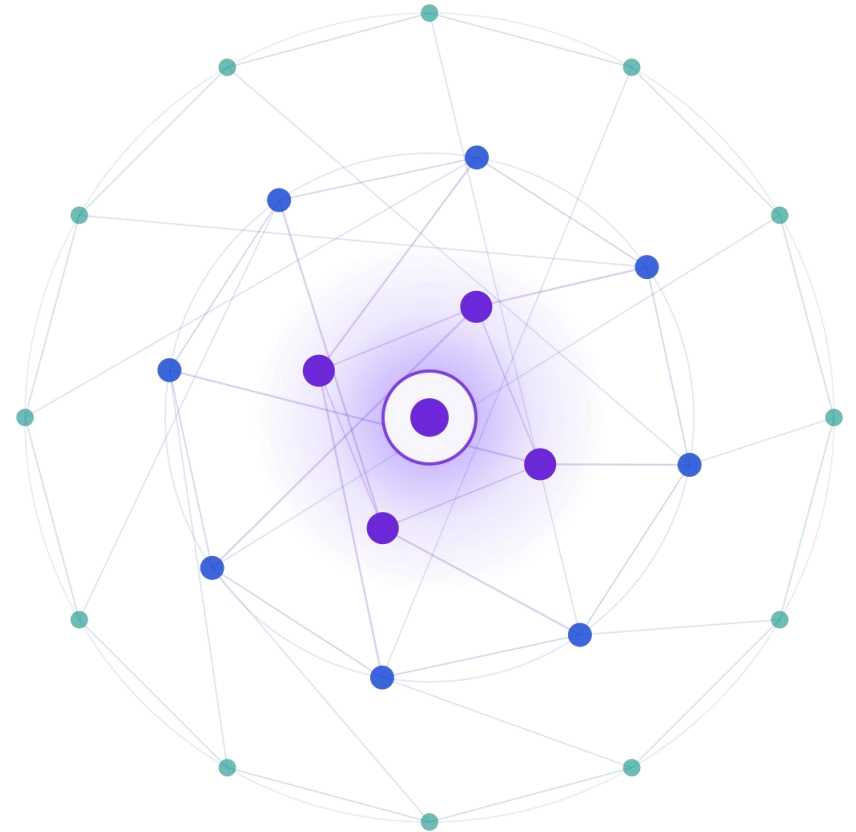


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 1 (Theme 1: Agentic Code Intelligence)
- Team Name - Code Alchemist
- College Name - Thapar Institute Of Engineering and Technology
- Member Name & Email 1- Kanika Sahni , ksahni_be24@thapar.edu
- Member Name & Email 2- Abhinandan Wadhwa , awadhwa_be24@thapar.edu
- Submission Github link - <https://github.com/kanika190905/litmus>

Project - Litmus: execution-verified, version-aware code retrieval



Theme

- **Theme 1 - Agentic Code Intelligence:** agents must find the right code before they can fix or extend it
- **Task:** given a code library and a natural-language query, rank the code snippets by relevance (retrieval only - generation is out of scope)
- **P0 Retrieval accuracy:** screened on the CoIR-apps test split with NDCG@10 and MRR (MTEB AppsRetrieval)
- **P1 Retrieval across versions:** rebuild indexes / caches for any version in reasonable time
- **Bonus Evolutionary retrieval:** rank across all versions, where snippets are near-identical
- **Constraint:** CPU, minimal GPU; faster than an LLM

What the benchmark really looks like

3,765 test queries = full competitive-programming statements (median 458 tokens; 17 in Russian)

8,765 Python solutions (5,000 belong to train problems = distractors)

exactly 1 relevant snippet per query;
identifiers carry almost no signal

Key observation: 98.4% of test queries contain sample tests (Input -> Output)

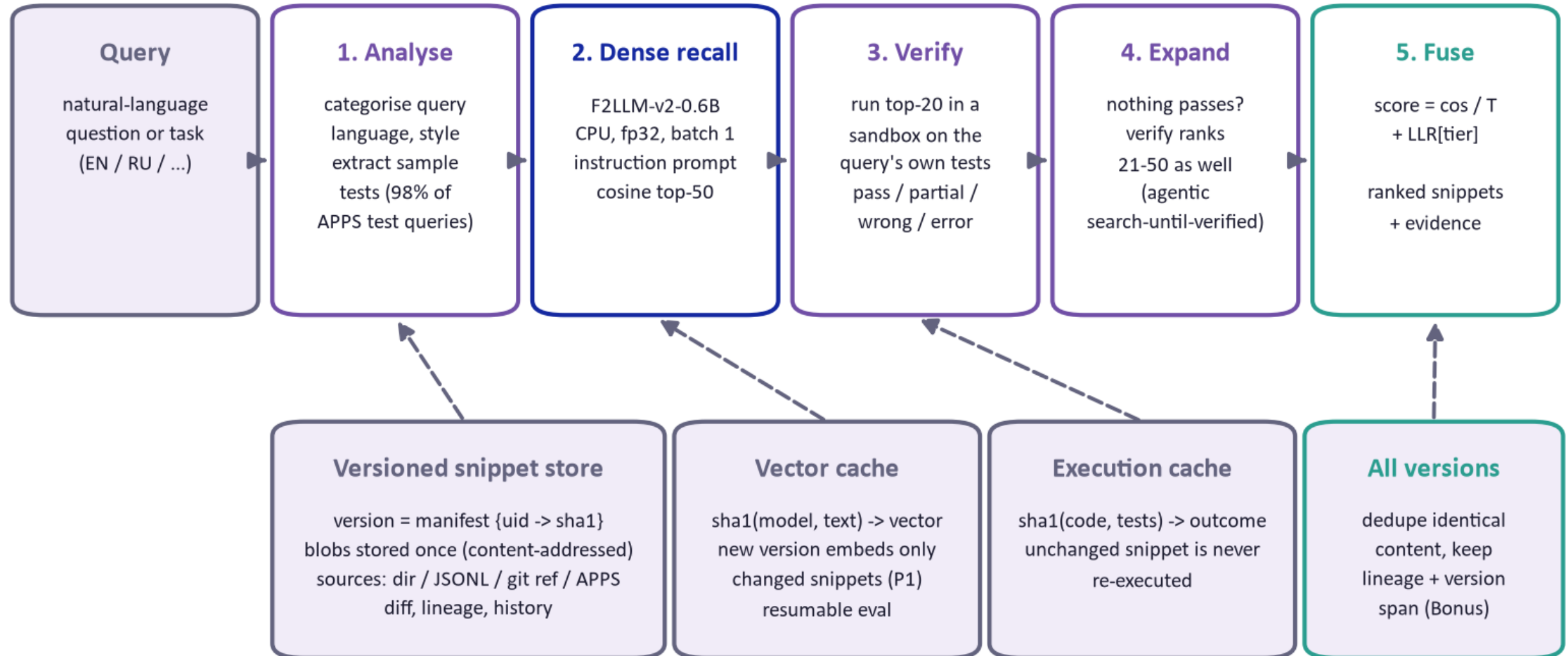
-> a candidate can be executed and checked, not just compared

Existing Solutions & Gaps

Approach (published MTEB AppsRetrieval)	NDCG@10	Gap
BM25 (lexical)	0.048	problem text and code share almost no words
General text embedders (OpenAI-3-large / e5-mistral-7B)	0.285 / 0.235	not trained for NL -> code semantics
Code-aware embedders, CPU-size (F2LLM-v2-0.6B)	0.905	ranks by similarity: a one-token bug looks identical
Large embedders / APIs (F2LLM-v2-8B, Gemini-embedding-2)	0.964 / 0.986	4-14B params or cloud API - not CPU / on-device
LLM re-ranking	-	slow, context-limited, needs GPU (per the theme brief)
Litmus (ours): F2LLM-v2-0.6B + execution verification, CPU	0.967	official run; above every published model up to 8B

- **Common gap:** every approach ranks by similarity only - none checks whether a snippet actually does what the query asks
- **Versions:** near-duplicate snippets (Bonus) are indistinguishable to embeddings; indexes are usually rebuilt from scratch per version (P1)

Our Solutions & Architecture Diagram



Demo & Product Walkthrough

Litmus

Execution-verified, version-aware code retrieval

codefuse ai/F2LLM-v2.0.68 - CPU - verify top-20 (+30)

Sample query (real CoRe-apps test queries)
q6018 — Little Artem got n stones on his birthday and nc ▼

Query

Little Artem got n stones on his birthday and now wants to give some of them to Masha. He knows that Masha cares more about the fact of receiving the present, rather than the value of that present, so he wants to give her stones as many times as possible. However, Masha remembers the last present she received, so Artem can't give her the same number of stones twice in a row. For example, he can give her 3 stones, then 1 stone, then again 3 stones, but he can't give her 3 stones and then again 3 stones right after that.

How many times can Artem give presents to Masha?

Store
apps-demo ▼

Version
v1 · 1000 snippets ▼

Top-k
10

☒ Execution verification (pass 2 & 3)

SearchCompare: dense-only vs verified

Query analysis

kind task_with_examples language en

style codeforces-style examples 4 verifiable yes

Example 1: input → expected output

1	1
---	---

Example 2: input → expected output

2	1
---	---

Example 3: input → expected output

3	2
---	---

Example 4: input → expected output

4	1
---	---

0 ms
embed query (cached)

0.5 ms
dense search · 1000 snippets

1 ms
verify 20 candidates

1
pass all examples

2 ms
total

view apps-demo#v1 · wrong-answer 4 · runtime-error 8 · can't-run 0

Ground-truth solution d6018: dense-only rank 4 → Litmus rank 1

#1 d6018 PASS 4/4 GROUND TRUTH dense #4 1 v1 cos 0.6574 · score 39.87 · history

n = int(input())
if n % 3 == 0:
 print(2 * (n // 3))
else:
 print(2 * (n // 3) + 1)

Example 1: pass 0.1 ms
got expected
1 1

Example 2: pass 0.1 ms
got expected
1 1

Example 3: pass 0.1 ms
got expected
2 2

Example 4: pass 0.1 ms
got expected
3 3

#2 d5287 PARTIAL 3/4 dense #1 1 v1 cos 0.7346 · score 38.23 · history

#3 d7781 PARTIAL 1/4 dense #2 1 v1 cos 0.6862 · score 35.81 · history

#4 d5151 PARTIAL 2/4 dense #3 1 v1 cos 0.6680 · score 34.90 · history

#5 d8364 WRONG ANSWER 0/4 dense #5 1 v1 cos 0.6325 · score 29.63 · history

#6 d8600 PARTIAL 2/4 dense #8 1 v1 cos 0.5078 · score 26.89 · history

#7 d7797 WRONG ANSWER 0/4 dense #6 1 v1 cos 0.5642 · score 26.21 · history

#8 d8254 PARTIAL 2/4 dense #12 1 v1 cos 0.4903 · score 26.01 · history

#9 d8623 PARTIAL 3/4 dense #13 1 v1 cos 0.4901 · score 26.01 · history

#10 d8103 RUNTIME ERROR 0/4 dense #7 1 v1 cos 0.5631 · score 25.15 · history

Litmus
codefuse ai/gpt4lm-v2-0.6B · CPU · verify-top-20 (+30)

Execution verified, version-aware code retrieval

Sample query (real CoLLM-app test queries)

q5287 [evolves] — Let's define a split of \$n\$ as a nonincreasing sequence of positive integers, the sum of which is \$n\$.

Query

Let's define a split of \$n\$ as a nonincreasing sequence of positive integers, the sum of which is \$n\$.

For example, the following sequences are splits of \$8\$: \$[4, 4]\$, \$[5, 3, 2]\$, \$[2, 2, 1, 1, 1, 1]\$, \$[5, 2, 1]\$.

The following sequences aren't splits of \$8\$: \$[1, 7]\$, \$[5, 4]\$, \$[5, 1]\$, \$\rightarrow [3, 5, 1, 1, 4, 1, 1]\$.

The weight of a split is the number of elements in the split that are equal to the first element. For example, the weight of the split \$[1, 1, 1, 1, 1]\$ is 5.

Store Version Top-k
 apps-demo ALL versions (evol) 6

☒ Execution verification (pass 2 & 3)

Search
Compare: dense-only vs verified

0 ms
embed query (cached)

0.5 ms
dense search - 1049 snippets

1 ms
verify 20 candidates

1
pass all examples

2 ms
total

view apps-demo@111 · wrong-answer 14 · runtime-error 2 · can't-run 0

#	id	status	ground truth	dense q#1	v#1	v#2	cos	score	history	
#1	d5287b005e018f	PASS 3/3	GROUND TRUTH	dense #3 : v1	v3		cos 0.5982	score 36.91	history	
#2	d6018b37525054	PARTIAL 1/3		dense #4 : v1	v2	v3	cos 0.5487	score 28.94	history	
#3	d778185afa715e	WRONG ANSWER 0/3		dense #1 : v1	v2	v3	cos 0.6869	score 28.35	history	
#4	d52873e09ae9e6	WRONG ANSWER 0/3	modified version of ground truth	dense #2 : v2			cos 0.6831	score 28.16	history	
#5	d8364b66a7209	WRONG ANSWER 0/3		dense #5	v1	v3	cos 0.5454	score 25.42	history	
#6	d6327b0cc4363f	WRONG ANSWER 0/3		dense #6	v1	v2	v3	cos 0.5425	score 25.13	history

Query analysis

kind task_with_examples language en

style codeforces-style
examples 3
verifiable yes

Example 1: input → expected output

```

1 2 3 4 5 6 7 8 9 10
2 3 4 5 6 7 8 9 10 11
3 4 5 6 7 8 9 10 11 12
4 5 6 7 8 9 10 11 12 13
5 6 7 8 9 10 11 12 13 14
6 7 8 9 10 11 12 13 14 15
7 8 9 10 11 12 13 14 15 16
8 9 10 11 12 13 14 15 16 17
9 10 11 12 13 14 15 16 17 18
10 11 12 13 14 15 16 17 18 19
11 12 13 14 15 16 17 18 19 20
12 13 14 15 16 17 18 19 20 21
13 14 15 16 17 18 19 20 21 22
14 15 16 17 18 19 20 21 22 23
15 16 17 18 19 20 21 22 23 24
16 17 18 19 20 21 22 23 24 25
17 18 19 20 21 22 23 24 25 26
18 19 20 21 22 23 24 25 26 27
19 20 21 22 23 24 25 26 27 28
20 21 22 23 24 25 26 27 28 29
21 22 23 24 25 26 27 28 29 30
22 23 24 25 26 27 28 29 30 31
23 24 25 26 27 28 29 30 31 32
24 25 26 27 28 29 30 31 32 33
25 26 27 28 29 30 31 32 33 34
26 27 28 29 30 31 32 33 34 35
27 28 29 30 31 32 33 34 35 36
28 29 30 31 32 33 34 35 36 37
29 30 31 32 33 34 35 36 37 38
30 31 32 33 34 35 36 37 38 39
31 32 33 34 35 36 37 38 39 40
32 33 34 35 36 37 38 39 40 41
33 34 35 36 37 38 39 40 41 42
34 35 36 37 38 39 40 41 42 43
35 36 37 38 39 40 41 42 43 44
36 37 38 39 40 41 42 43 44 45
37 38 39 40 41 42 43 44 45 46
38 39 40 41 42 43 44 45 46 47
39 40 41 42 43 44 45 46 47 48
40 41 42 43 44 45 46 47 48 49
41 42 43 44 45 46 47 48 49 50
42 43 44 45 46 47 48 49 50 51
43 44 45 46 47 48 49 50 51 52
44 45 46 47 48 49 50 51 52 53
45 46 47 48 49 50 51 52 53 54
46 47 48 49 50 51 52 53 54 55
47 48 49 50 51 52 53 54 55 56
48 49 50 51 52 53 54 55 56 57
49 50 51 52 53 54 55 56 57 58
50 51 52 53 54 55 56 57 58 59
51 52 53 54 55 56 57 58 59 60
52 53 54 55 56 57 58 59 60 61
53 54 55 56 57 58 59 60 61 62
54 55 56 57 58 59 60 61 62 63
55 56 57 58 59 60 61 62 63 64
56 57 58 59 60 61 62 63 64 65
57 58 59 60 61 62 63 64 65 66
58 59 60 61 62 63 64 65 66 67
59 60 61 62 63 64 65 66 67 68
60 61 62 63 64 65 66 67 68 69
61 62 63 64 65 66 67 68 69 70
62 63 64 65 66 67 68 69 70 71
63 64 65 66 67 68 69 70 71 72
64 65 66 67 68 69 70 71 72 73
65 66 67 68 69 70 71 72 73 74
66 67 68 69 70 71 72 73 74 75
67 68 69 70 71 72 73 74 75 76
68 69 70 71 72 73 74 75 76 77
69 70 71 72 73 74 75 76 77 78
70 71 72 73 74 75 76 77 78 79
71 72 73 74 75 76 77 78 79 80
72 73 74 75 76 77 78 79 80 81
73 74 75 76 77 78 79 80 81 82
74 75 76 77 78 79 80 81 82 83
75 76 77 78 79 80 81 82 83 84
```

Across all versions (Bonus)

q5287: v2 regression 'n // 2' -> 'n // 3' is dense #2; it fails 0/3 examples and drops to #4.

The correct content (identical in v1 and v3) ranks #1 with 3/3 passing.

- **Run:** `litmus serve -> http://127.0.0.1:8000` | CLI: `litmus search @demo/queries/q5287.txt --store apps-demo --version all`

Tools and tech stack used

Retrieval model

codefuse-ai/F2LLM-v2-0.6B (Apache-2.0)
sentence-transformers + PyTorch CPU
fp32
batch-1 CPU inference (2.7x faster)

Evaluation

MTEB 2.21 - AppsRetrieval
HF datasets (ColR-apps, pinned revision)
own NDCG / MRR / Recall checks

Execution sandbox

persistent Python worker pool
PEP 578 audit hooks, fd redirection
Job Objects / RLIMIT_AS, watchdog

Versioned storage

content-addressed SQLite caches
JSON manifests (git-tree style)
ingest: dir, JSONL, git ref, AST chunks

Product

FastAPI + Uvicorn API
single-page web UI (no build step)
`litmus` CLI

Engineering

pytest suite (fake embedder)
Python 3.12, Windows + Linux
Kaggle GPU for the one-off official benchmark run

Impact & Use case

- **AI coding agents:** fetch code that is proven to behave as asked before editing it - fewer wrong-file edits
- **Regression hunting:** 'which version still passes this failing example?' - search across versions by behaviour
- **On-device / air-gapped code search:** CPU only, no code leaves the machine - fits internal code bases
- Incremental indexing for fast-moving repos: a commit costs only the changed functions
- **Explainable ranking:** every result shows the evidence (outputs vs expected), not just a score

0.903 -> 0.967

official NDCG@10 on the full test split: same 0.6B model alone -
> with Litmus

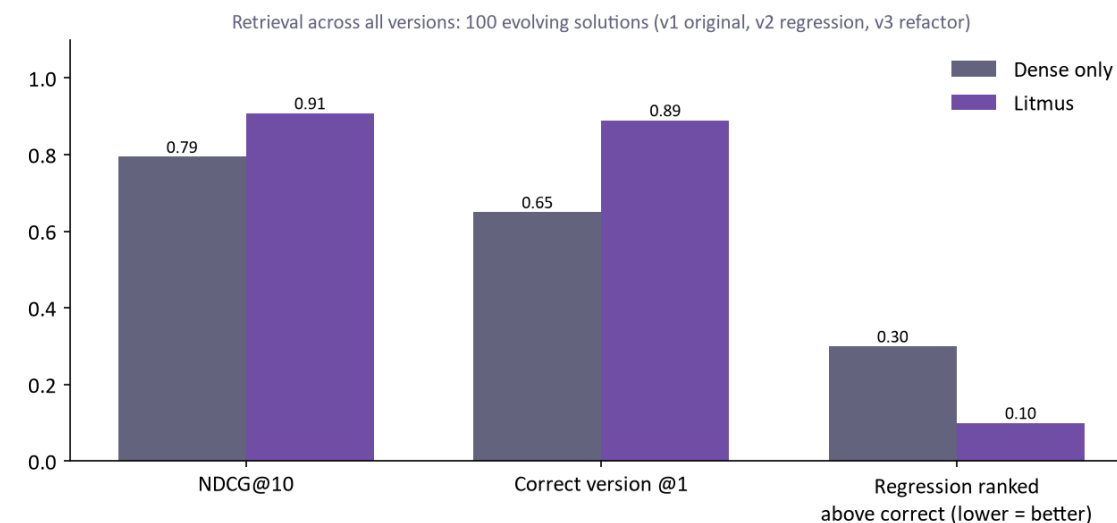
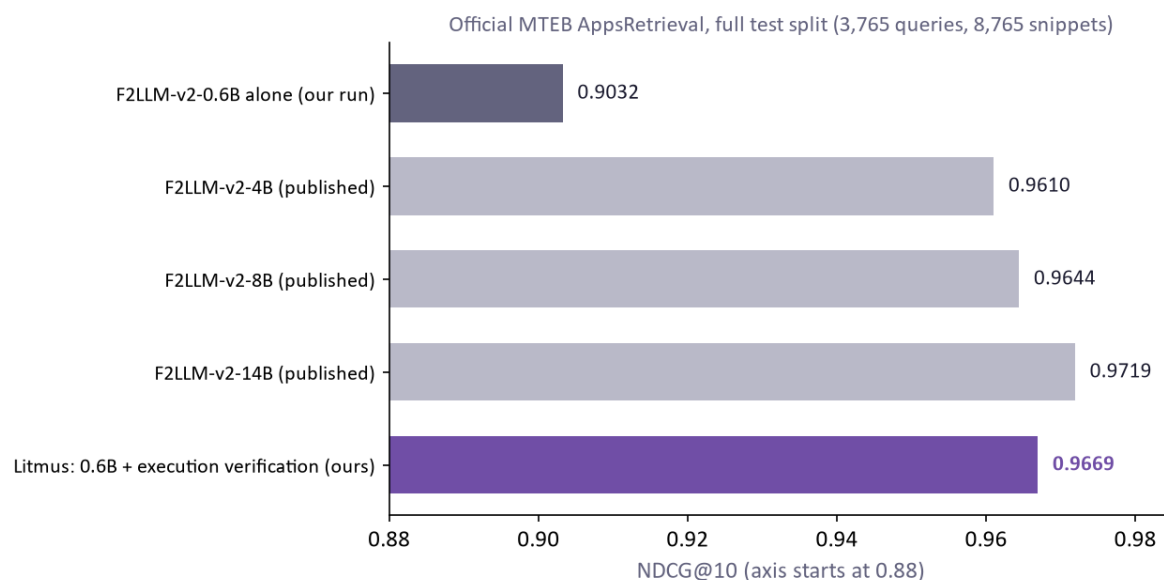
30.0% -> 10.0%

buggy version ranked above the correct one (all-versions search)

50 of 995

snippets re-embedded to index a new version (only the changed ones)

Innovation highlights, results and limitations



Limitations (honest)

Official full test split: NDCG@10 0.9669 / MRR@10 0.9604 (dense-only 0.9032). Embedding for this one-off run used a Kaggle GPU; same fp32 model, the system itself runs on CPU

Leaderboard models are search-only; Litmus adds a verification step, so it is a pipeline, not a single embedding model

Fusion weights are hand-set priors (not fitted); queries without sample tests fall back to dense ranking

What's next

- **Multiple-answer problems:** task-specific checkers so 'print any valid answer' solutions can pass
- **Learned fusion:** fit the evidence weights on the ColR train split (5,000 queries) instead of hand-set priors
- **More evidence:** function-level harnesses (Solution().method calls) and generated extra test inputs
- **Diff-aware queries:** 'which commit changed the parsing of X?' using version deltas
- **Scale-out:** HNSW index, OpenVINO / ONNX int8 with calibration, container sandbox (gVisor)
- **IDE / agent integration:** expose search + verify as a tool for coding agents

Brownie points slide(differentiation)

Verify, don't just match

Executes candidates on the query's own examples. Gold solution passes all its examples in 90.8% of 600 sampled test queries; 1 of 3,000 random snippets do.

Versions by construction

Git-tree-like manifests + content-addressed vectors and execution results: a new version costs only what changed; reverts cost nothing.

Evolution-aware ranking

Near-identical versions are separated by behaviour, not text; regressions drop below correct versions.

CPU-first, measured

bf16 -> fp32, batch-1 (2.7x), persistent sandbox workers (ms per check); int8 rejected on measured fidelity.

Robust to real data

Russian statements, AtCoder/Codeforces/CodeChef formats, Python-2 and function-body snippets, numpy.

Honest and reproducible

Official MTEB path (AbsEncoder + SearchProtocol), every number scripted, pending items labelled, AI disclosure.

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y/N) -> **Y** - <https://github.com/kanika190905/litmus>
- README with reproducible setup instructions (Y/N) -> **Y**
- Demo video, max 5 minutes (YouTube or Drive link) -> <https://github.com/kanika190905/litmus/blob/main/demo/Video%20Project.mp4>
- Presentation file (PPT or PDF) (Y/N) -> **Y**
- AI disclosure (AI_DISCLOSURE.md) -> **Y**
- APK / SDK -> **N/A - Python package + web app, no mobile component**
- Release v1.0.0 + MTEB result JSON -> **Y** - [results/appsretrieval_results.json](#)

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India